

Decoupled Image-to-Video Generation via Latent Flow: An Investigation into Embedding-Conditioned Rendering

Anonymous COMP4471 submission

Paper ID *****

Abstract

001 *Image-to-video generation from a single static image and*
002 *an action label requires simultaneously understanding both*
003 *what is in the scene and how it should evolve over time.*
004 *We investigate a decoupled framework inspired by the Joint-*
005 *Embedding Predictive Architecture (JEPA): a causal Trans-*
006 *former (Flow Model) predicts the trajectory of CLIP em-*
007 *beddings in a compact 512-dimensional semantic space,*
008 *and a conditioned UNet (Renderer) converts each predicted*
009 *embedding back to a pixel frame. We conduct a system-*
010 *atic set of experiments across four Renderer architectures—*
011 *FiLM conditioning, cross-attention to CLIP patch tokens,*
012 *contrastive conditioning loss, and optical flow prediction—*
013 *and find that each variant collapses to a mode where the em-*
014 *bedding condition is effectively ignored and the output is a*
015 *near-identity transformation of the input frame. We present*
016 *a detailed failure analysis identifying the root causes: van-*
017 *ishing embedding motion signal between adjacent frames,*
018 *a training objective fundamentally misaligned with condi-*
019 *tioning sensitivity, and the inherent difficulty of encoding*
020 *spatial motion into a single global feature vector. Despite*
021 *the negative result, our analysis highlights key design chal-*
022 *lenges for embedding-conditioned video synthesis and sug-*
023 *gests concrete alternative approaches.*

024 1. Introduction

025 Image-to-video (I2V) generation aims to produce a tempo-
026 rally coherent video sequence from a single static frame.
027 When conditioned on an action label (e.g., “walking”), the
028 generated video should depict natural motion consistent
029 with that action. This task is relevant to content creation,
030 video editing, and synthetic data augmentation for action
031 recognition systems.

032 Recent large-scale diffusion models such as Stable Video
033 Diffusion and Wan2.2 have demonstrated impressive results
034 by inflating image diffusion architectures into the tempo-
035 ral domain. However, these end-to-end approaches incur

substantial computational costs and remain prone to motion
flickering and identity drift.

Inspired by the JEPA framework, we investigate an al-
ternative: decoupling the problem into a **Flow Model** that
predicts motion in a semantic embedding space and a **Ren-**
derer that converts semantic predictions back to pixels. The
key hypothesis is that motion dynamics are easier to model
in a compact, abstract embedding space than in raw pix-
els. In an ideal scenario, the Flow Model would learn high-
level action dynamics (“a walking person alternates legs”),
and the Renderer would faithfully translate these semantic
blueprints into frames.

We implement this framework using frozen CLIP ViT-
B/32 encoders, a causal Transformer for embedding trajec-
tory prediction, and a UNet-based Renderer with condition-
ing injected via FiLM, cross-attention, or explicit flow pre-
diction heads. We systematically ablate the conditioning
mechanism and loss function across four Renderer variants.

Our key finding is negative but instructive: across all
four variants, the Renderer learns to ignore the embedding
condition and produces near-identical outputs regardless of
input conditioning. The root cause is not any single archi-
tectural choice but a fundamental tension between (1) the
minimal variation in CLIP embeddings between adjacent
video frames, and (2) loss functions that penalize deviation
from a nearly-static ground truth.

We present a detailed failure analysis quantifying the
embedding signal strength, the collapse behavior across
architectures, and the training dynamics. We believe
these findings provide useful guidance for future work on
embedding-conditioned generative models.

2. Related Work

2.1. Image-to-Video Generation

Early video generation methods employed recurrent net-
works or generative adversarial networks. More recently,
diffusion-based approaches have become standard. Stable
Video Diffusion fine-tunes a pre-trained image diffusion
model on video data by adding temporal layers. Wan2.2 fol-

074 lows a similar approach at larger scale. These models gener-
075 ally operate in a VAE latent space, simultaneously modeling
076 spatial structure and temporal dynamics in one pipeline. We
077 experiment with a fundamentally different architecture that
078 attempts to factor these responsibilities across two modules.

079 2.2. Latent Predictive Architectures

080 The JEPA framework proposes that predictive learning
081 should operate in representation space rather than input
082 space. V-JEPA applies this principle to self-supervised
083 video learning by predicting masked spatio-temporal fea-
084 tures. Our Flow Model follows the same intuition: pre-
085 dict the trajectory of CLIP embeddings rather than raw pix-
086 els. Where our work differs is the downstream rendering
087 step—converting predicted embeddings back to pixels—
088 which proves to be the critical bottleneck.

089 2.3. Conditioned Image Generation

090 Feature-wise Linear Modulation (FiLM) and cross-
091 attention are standard techniques for conditioning image
092 generation on external information. FiLM applies channel-
093 wise affine transformations $\gamma(c) \odot x + \beta(c)$ predicted from
094 a conditioning vector c , and has been widely used in vi-
095 sual question answering, style transfer, and video predic-
096 tion. Cross-attention, popularized by the Transformer ar-
097 chitecture, allows each spatial position in a feature map to
098 attend to a set of conditioning tokens. Our experiments test
099 both mechanisms independently and in combination, and
100 find that neither is sufficient when the conditioning signal
101 (adjacent-frame CLIP embeddings) varies by less than 3%
102 in cosine distance.

103 2.4. Optical Flow and Warping

104 Frame interpolation and video prediction often employ ex-
105 plicit optical flow prediction followed by warping. Methods
106 such as RAFT achieve state-of-the-art optical flow estima-
107 tion through iterative refinement, while simpler approaches
108 use Farneback or TV-L1 flow as a supervision signal. Our
109 final Renderer variant adopts a flow-based approach: a
110 UNet backbone predicts a dense flow field conditioned on
111 CLIP embeddings, and the previous frame is warped via
112 differentiable bilinear sampling. We use Farneback optical
113 flow as pseudo-supervision and apply total variation regu-
114 larization to encourage spatial smoothness.

115 2.5. Action-Conditioned Video Generation

116 Motion generation conditioned on action labels has been
117 studied in controlled settings such as video game environ-
118 ments and simplified human motion benchmarks. Text-to-
119 video models such as Make-A-Video and Imagen Video
120 use large language model embeddings for open-vocabulary
121 conditioning at scale. Our setting is intermediate: we use

closed-set action labels encoded through CLIP text embed-
dings, providing a clean semantic signal in the same embed-
ding space as the visual features.

3. Data

We use the HMDB51 dataset, containing 6,754 video
clips across 51 human action categories such as “walking,”
“drinking,” “brushing hair,” and “climbing stairs.” Videos
average approximately 3 seconds in duration at 25–30 fps.
The dataset is split into training (70%) and test (30%) sets
following the first official split.

3.1. Preprocessing

For each video, we sample 17 consecutive frames ($n = 16$
prediction steps) at a stride of 2, and resize all frames to
 224×224 pixels. CLIP embeddings (CLS token + 49 patch
tokens) are pre-extracted for the entire dataset (242MB) to
avoid redundant forward passes during Flow Model train-
ing. The Renderer samples random frame pairs (I_{t-1}, I_t)
from each video with a gap of 1 frame.

3.2. Embedding Analysis

A key diagnostic we perform early is measuring the varia-
tion in CLIP embeddings across frames. Table 1 summa-
rizes pairwise cosine similarities.

Table 1. CLIP ViT-B/32 embedding cosine similarity. Same-video
frames are highly similar ($1 - \cos = 0.024$ for adjacent frames),
limiting the information available for frame-to-frame motion pre-
diction.

Comparison	Mean cos
Same video, distant frames ($\Delta t \approx 60$)	0.862
Same video, adjacent frames ($\Delta t = 1$)	0.976
Different videos, same action class	0.664
Different action classes	0.564

The critical observation is that adjacent-frame embed-
dings differ by only $1 - \cos \approx 0.024$ on average. This means
the embedding trajectory that the Flow Model must predict,
and that the Renderer must use as a conditioning signal,
contains very little per-frame variation. Any learning algo-
rithm must amplify this tiny signal by orders of magnitude
to produce meaningful pixel-space changes.

4. Methods

Our framework consists of three frozen encoders and two
trainable modules. Figure 1 illustrates the overall pipeline.

4.1. Encoders

Image Encoder. Frozen CLIP ViT-B/32. We extract both
the 512-dim CLS embedding E_t and the 49 spatial patch to-

Pipeline Overview.

$$\begin{aligned}
& [\text{Input Image } I_0] \rightarrow [\text{CLIP ViT}] \rightarrow [E_0] \\
& [\text{Action Label}] \rightarrow [\text{CLIP Text}] \rightarrow [L] \\
& [L, E_0] \rightarrow [\textbf{Flow Model: Causal Transformer}] \rightarrow [\hat{E}_1, \dots, \hat{E}_n] \\
& [I_{t-1}, \hat{E}_t, L, \text{patches}] \rightarrow [\textbf{Renderer: FiLM + Cross-Attn UNet}] \rightarrow [I_t]
\end{aligned}$$

The Flow Model and Renderer are trained separately. At inference, the Flow Model generates the embedding trajectory autoregressively; the Renderer produces each frame conditioned on the predicted embedding.

Figure 1. Overall pipeline. Frozen CLIP encoders produce image and label embeddings. The Flow Model predicts the embedding trajectory. The Renderer produces each output frame conditioned on the predicted embedding and the previous frame.

kens (7×7 grid, 512-dim each) from the final vision transformer layer.

Label Encoder. Frozen CLIP text encoder. Action labels are converted from directory names to natural text (e.g., “brush hair”) and encoded to 512-dim semantic embeddings L .

4.2. Flow Model

A 4-layer causal Transformer encoder (12.9M parameters) with 8 attention heads and feedforward dimension 2048. The input sequence is $[L, E_0, E_1, \dots, E_{n-1}]$, and the model is trained with shifted teacher forcing: position t predicts $\hat{E}_t$ from $(L, E_0, \dots, E_{t-1})$.

We compare two loss functions:

- **MSE:** $\mathcal{L} = \frac{1}{n} \sum_t \|\hat{E}_t - E_t\|_2^2$
- **Cosine:** $\mathcal{L} = \frac{1}{n} \sum_t (1 - \cos(\hat{E}_t, E_t))$

4.3. Renderer

The Renderer is a UNet (4 down/up blocks, 40–43M parameters depending on variant) that takes I_{t-1} as input and produces an output conditioned on $\hat{E}_t$, L , and patch tokens. We experiment with four variants:

1. **FiLM (baseline):** channel-wise Feature-wise Linear Modulation from a condition vector $[\hat{E}_t; L] \in \mathbb{R}^{1024}$, applied at each residual block. Output: pixel residual ΔI_t . Loss: $L1(\Delta I_t, I_t - I_{t-1})$.
2. **FiLM + Cross-Attention:** FiLM conditioning plus multi-head cross-attention from UNet feature maps to CLIP patch tokens at the bottleneck and first two upsampling blocks. Output: pixel residual ΔI_t . Loss: L1.
3. **FiLM + Cross-Attention + Contrastive:** same as variant 2, plus a contrastive conditioning loss: for a batch, we compute $\mathcal{L}_{\text{cont}} = \|\cos(\Delta I_{\text{real}}, \Delta I_{\text{fake}}) - \cos(E_{\text{real}}, E_{\text{fake}})\|_2^2$, where fake conditions are shuffled embeddings from other videos in the batch. This explicitly penalizes the Renderer for producing identical outputs from different conditions.

4. **Flow Prediction:** same UNet backbone with FiLM + cross-attention, but the output head predicts a dense optical flow field $\in \mathbb{R}^{2 \times 224 \times 224}$. The previous frame is warped via the predicted flow: $I_t = \text{warp}(I_{t-1}, \text{flow})$. Loss: $L1(I_t^{\text{warped}}, I_t^{\text{gt}}) + \lambda_{\text{TV}} \cdot \text{TV}(\text{flow}) + \lambda_{\text{FB}} \cdot L1(\text{flow}, \text{Farneback})$.

4.4. Training Details

All models are trained with AdamW ($lr = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}) and a cosine annealing learning rate schedule over 100 epochs. We use a batch size of 8 with gradient accumulation over 2 steps (effective batch size 16). Gradient clipping with maximum norm 1.0 is applied. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8GB VRAM). Complete training of the Flow Model takes approximately 4 hours; each Renderer variant takes approximately 7 hours.

Flow Model training data pipeline. Because passing CUDA models through PyTorch DataLoader workers is not supported on Windows, we pre-extract CLIP embeddings for all 6,754 videos (17 frames each) and save them to disk as a single 242MB file. The Flow Model training loop loads pre-extracted embeddings directly, requiring no CLIP forward passes during training. This eliminates the DataLoader CUDA issue and accelerates training by approximately $3\times$.

Renderer training data pipeline. The Renderer uses HMDB51FrameDataset, which loads raw frames from disk (via OpenCV) and performs image transformations in the DataLoader workers. CLIP encoding happens in the main training loop where CUDA is available. Farneback optical flow is computed on CPU using OpenCV, adding approximately 30% overhead per batch.

4.5. Implementation Notes

Shifted teacher forcing. An earlier version of the Flow Model used “zero-placeholder” inputs where the model received $[L, E_0, 0, \dots, 0]$ and was expected to predict all fu-

ture embeddings in a single forward pass. This created a train-inference mismatch: at inference time, the model sees its own predictions rather than zeros. Shifted teacher forcing, where the model receives the real embedding sequence and predicts one step ahead at each position, aligns training and inference exactly. We found this improved embedding trajectory quality significantly (from 2.0% to 82.5% of GT motion, combined with the cosine loss change).

Multi-scale flow prediction. In the flow prediction Renderer variant, we considered predicting flow at multiple decoder resolutions and combining them via coarse-to-fine refinement (similar to RAFT). However, initial experiments showed that even a single-scale flow head struggled to produce consistent non-zero outputs, so we deferred multi-scale investigation to future work.

5. Experiments

5.1. Evaluation Protocol

We evaluate using the following metrics:

- **Embedding Trajectory MSE:** MSE between predicted and ground-truth CLIP embedding sequences.
- **Inter-frame 1-cos Distance:** $1 - \cos(\hat{E}_t, \hat{E}_{t+1})$, measuring how much “motion” the predicted embeddings contain.
- **Conditioning Sensitivity:** $\frac{\| \text{Renderer}(I, E_t) - \text{Renderer}(I, E_0) \|_1}{\| \text{Renderer}(I, E_0) \|_1 + \epsilon}$, measuring whether changing the embedding changes the Renderer output.

5.2. Flow Model Results

Table 2 summarizes the Flow Model results.

Table 2. Flow Model performance with MSE vs. cosine similarity loss. Cosine loss prevents embedding collapse while maintaining prediction accuracy.

Metric	MSE Loss	Cosine Loss
Train loss (final)	0.0116	0.0264
Inter-frame 1-cos (GT)	0.0290	0.0290
Inter-frame 1-cos (pred)	0.0006	0.0239
% of GT motion preserved	2.0%	82.5%

The MSE-trained Flow Model achieves low MSE (0.0116) but collapses to predicting nearly identical embeddings at every timestep (1-cos $\approx$ 0.0006). Cosine loss resolves this: predicted embeddings retain 82.5% of the ground-truth motion variation while maintaining comparable embedding quality (MSE between predicted and GT embeddings actually improves in terms of cosine distance).

5.3. Renderer Results

Table 3 presents the core negative result: across all four architectures, the Renderer learns to ignore the embedding

condition. 266

Table 3. Renderer conditioning sensitivity across four architecture variants. A ratio of ~ 1.0 indicates the model produces identical outputs regardless of the embedding condition.

Renderer Variant	Sensitivity Ratio
1. FiLM + L1	1.03x
2. FiLM + Cross-Attn + L1	0.98x
3. FiLM + Cross-Attn + L1 + Contrastive	0.98x
4. FiLM + Cross-Attn + Flow Pred + Rec + TV + FB	1.01x

For all variants, the conditioning sensitivity ratio is approximately 1.0, meaning the magnitude of the Renderer’s output does not meaningfully change when the conditioning embedding changes. In variants 1–3 (residual prediction), the absolute output magnitude converges to ≈ 0.005 —essentially near-zero. In variant 4 (flow prediction), the predicted flow field converges back to zero after initial non-zero activity from random initialization. 267 268 269 270 271 272 273 274

5.4. Diagnostic Experiments

To isolate the failure mode, we conduct several diagnostic tests: 275 276 277

Embedding distance test. We feed the Renderer embeddings from frames at varying temporal distances ($[t, t + 1, t + 4, t + 8, t + 16]$) and measure the output magnitude. Even with frame $t + 16$ (1-cos = 0.31, a meaningful change), the output magnitude remains unchanged from the $t + 1$ case (1-cos = 0.03). This confirms that the model is not merely limited by the weak signal in adjacent frames—it has learned to ignore the embedding entirely. 278 279 280 281 282 283 284 285

Farneback flow as pseudo-target. We compute Farneback optical flow between adjacent frames (mean magnitude 0.39 px, max 2.84 px) and use it as a supervised target. The flow head initially produces non-zero flow but converges back to zero as the reconstruction and smoothness losses dominate. 286 287 288 289 290 291

CLIP feature discrimination. We verify that CLIP embeddings *do* contain frame-to-frame information: the cosine similarity gap between same-video adjacent frames (0.976) and different videos of the same class (0.664) is 0.312—a substantial difference. The information exists, but translating it from a 512-dim global vector into spatially meaningful pixel changes proves fundamentally difficult with our architectures. 292 293 294 295 296 297 298 299

6. Discussion: Why Did It Fail? 300

We identify the following contributing factors, ordered from most fundamental to most specific. 301 302

303	6.1. Primary Cause: CLIP Embeddings Lack the	
304	Necessary Information	
305	We believe this is the root cause from which all other issues derive. CLIP ViT-B/32 is trained with a contrastive image-text matching objective: given an image and a text description, maximize the cosine similarity of their embeddings. This objective encourages the model to extract <i>semantic content</i> —“what is in this image”—and compresses it into a compact 512-dimensional vector.	
312	What CLIP’s training objective does <i>not</i> require is preserving the kind of information needed for frame-to-frame rendering:	
315	• Spatial configuration: The precise 2D coordinates of body parts, objects, and their relative positions.	
317	• Pose and articulation: The joint angles of limbs, the orientation of a head, the bend of a knee.	
319	• Dense correspondence: Which pixel in frame t corresponds to which pixel in frame $t + 1$.	
321	A CLIP embedding can tell you “this is a person walking” with high confidence, but it cannot tell you “the right leg is currently forward at a 30-degree angle, positioned at pixel coordinates (120, 180).” The latter is precisely the information a Renderer needs to predict what the next frame should look like.	
327	This explains the asymmetry in our results: the Flow Model succeeded because predicting an embedding trajectory only requires matching the statistical distribution of CLIP embeddings—a task CLIP embeddings contain sufficient information for. The Renderer failed because decoding an embedding back to a specific spatial configuration of pixels requires information that was discarded during CLIP’s encoding process.	
335	Empirically, we confirmed this information bottleneck: adjacent frames of the same video have CLIP embeddings with cosine similarity ≈ 0.976 ($1 - \cos \approx 0.024$). This tiny difference must encode the entire motion transformation from one frame to the next—a compression ratio far exceeding what any decoder architecture can plausibly invert.	
342	6.2. Contributing Factor: Loss Function Misalignment	
344	All our loss functions (L1 on residuals, L1 on reconstruction after warping) have an optimal trivial solution near the identity transformation: $\Delta I_t \approx 0$ and $\text{flow} \approx 0$. When the conditioning signal carries almost no usable information (as argued above), identity mapping becomes the rational optimum.	
350	6.3. Contributing Factor: FiLM Cannot Express Spatial Guidance	
352	Feature-wise Linear Modulation applies channel-wise scaling and shifting uniform across all spatial positions. Even if	
354	the embedding <i>did</i> contain spatial information, FiLM lacks the mechanism to route it to the correct spatial location. Cross-attention to CLIP patch tokens partially addresses this routing problem, but the patch tokens themselves still lack dense correspondence information due to the primary cause above.	
360	6.4. Limitations of Our Study	
361	We acknowledge several limitations: (1) we only explored CLIP ViT-B/32 as the embedding backbone—larger ViT variants or dense-prediction models like DINOv2 may have different properties; (2) we did not attempt joint end-to-end training of the Flow Model and Renderer, which could allow the Flow Model to learn embeddings specifically optimized for rendering; (3) we used a relatively small dataset (6.7k videos), and the collapse behavior may differ at larger scale where more diverse motion patterns provide a stronger learning signal; (4) HMDB51 videos are constrained to specific human action categories, and the learned dynamics may not generalize to more open-ended video generation.	
373	6.5. What We Learned	
374	Despite the negative result, this investigation yielded several lessons that may benefit future work in this area:	
376	Embedding choice is critical. The choice of CLIP ViT-B/32 as our embedding backbone was motivated by its joint vision-language space and compact dimensionality. However, CLIP embeddings are optimized for semantic discrimination, not spatial correspondence. For a decoupled I2V pipeline to succeed, the embedding space must encode both <i>what</i> is in the scene and <i>where</i> things are spatially—properties that CLIP’s contrastive training objective does not guarantee.	
385	Loss functions define the optimization landscape. We observed that all tested pixel-space loss functions (L1, L1 + TV, L1 + Farneback) contain a near-identity trivial solution ($\Delta I \approx 0$, $\text{flow} \approx 0$). This is not specific to our architecture—any model trained with a pixel reconstruction loss on adjacent-frame pairs faces the same challenge. Loss functions that operate in a learned feature space (perceptual loss, adversarial loss) may provide a stronger signal for motion learning.	
394	The JEPA philosophy needs a rendering pathway. JEPA’s core insight—predict in representation space, not input space—proved effective for our Flow Model (82.5% of GT motion). But the companion challenge of <i>decoding</i> those predictions back to the pixel domain is equally hard and received comparatively little attention in our implementation. The rendering problem deserves its own careful treatment rather than being treated as an architectural afterthought.	

403

6.6. Future Work

404

Several promising directions emerge from this analysis.

405

Dense spatial embeddings. Replacing CLIP with

406

a model trained for dense visual correspondence—such

407

as DINOv2, Stable Diffusion’s VAE latents, or a self-

408

supervised optical flow network—could provide embed-

409

dings that naturally encode spatial configuration. A tra-

410

jectory in DINOv2 patch feature space would carry richer

411

spatial information than a trajectory in CLIP CLS space.

412

Joint end-to-end training. Training the Flow Model

413

and Renderer jointly, with gradients flowing from the ren-

414

dering loss back through the Flow Model, would allow the

415

embedding space to adapt to the needs of the Renderer.

416

The Flow Model would no longer be constrained to match

417

CLIP’s embedding structure and could learn features specifi-

418

cally optimized for frame rendering.

419

Iterative refinement. Rather than predicting the full

420

frame transformation in a single forward pass, a multi-step

421

refinement process (similar to diffusion models or recurrent

422

refinement) could gradually build up the output from an ini-

423

tial near-identity estimate. This would allow the model to

424

learn motion in stages—coarse structure first, then fine de-

425

tails.

426

Adversarial and perceptual losses. GAN-based losses

427

or CLIP-based perceptual losses directly penalize lack of

428

motion and unrealistic static output, potentially breaking

429

the near-identity local minimum that trapped all of our L1-

430

trained models.

431

Explicit motion field supervision. Using a pre-trained

432

optical flow network (e.g., RAFT) instead of Farneback as

433

a supervision target could provide a cleaner learning signal.

434

Combined with a rendering loss, this could guide the model

435

toward physically plausible motion while maintaining vi-

436

sual fidelity.

437

7. Conclusion

438

We set out to build a decoupled image-to-video genera-

439

tion system in which a causal Transformer predicts action-

440

conditioned embedding trajectories and a conditioned UNet

441

renders those trajectories to pixels. The Flow Model com-

442

ponent was partially successful: with cosine similarity

443

loss it learned to predict embedding trajectories preserving

444

82.5% of ground-truth motion variation, while MSE loss

445

led to complete collapse. However, the Renderer failed

446

across all four conditioning architectures tested: FiLM,

447

cross-attention, contrastive conditioning, and optical flow

448

prediction.

449

Our analysis suggests the primary cause is a fundamen-

450

tal information bottleneck: CLIP ViT-B/32 embeddings are

451

trained for semantic image-text alignment and do not pre-

452

serve the spatial configuration, pose, and correspondence

453

information necessary for pixel-level frame rendering. Ad-

454

jacent frames differ by less than 3% in CLIP embedding

455

cosine distance, an information compression ratio that is

456

implausible for any decoder architecture to invert. All

457

other observed problems—loss function collapse to identity,

458

FiLM’s inability to express spatial motion—can be under-

459

stood as consequences of this root cause.

460

Future directions suggested by this analysis include:

461

(1) replacing CLIP with a dense visual representation

462

trained for spatial correspondence, such as DINOv2 patch

463

features, Stable Diffusion VAE latents, or self-supervised

464

optical flow features; (2) training the Flow Model and Ren-

465

derer jointly end-to-end, so the embedding space can adapt

466

to the specific needs of rendering rather than being con-

467

strained by a frozen pre-trained encoder; (3) adopting losses

468

that explicitly penalize lack of motion, such as adversar-

469

ial or perceptual losses, to break the near-identity local

470

minimum; (4) incorporating explicit motion representations

471

such as optical flow or keypoint trajectories as intermediate

472

supervision targets rather than relying solely on the embed-

473

ding to carry all motion information.

474

References

475

[1] A. Blattmann, T. Dockhorn, S. Kulal, et al. “Stable Video

476

Diffusion: Scaling Latent Video Diffusion Models to Large

477

Datasets.” *arXiv:2311.15127*, 2023.

478

[2] Alibaba. “Wan2.2: Large-Scale Image-to-Video Genera-

479

tion.” 2025–2026.

480

[3] Y. LeCun. “A Path Towards Autonomous Machine Intelli-

481

gence.” *OpenReview*, 2022.

482

[4] A. Bardes, Q. Garrido, J. Ponce, et al. “Revisiting Feature

483

Prediction for Learning Visual Representations from Video.”

484

arXiv:2404.08471, 2024.

485

[5] A. Radford, J. W. Kim, C. Hallacy, et al. “Learning Trans-

486

ferable Visual Models From Natural Language Supervision.”

487

ICML, 2021.

488

[6] H. Kuehne, H. Jhuang, E. Garrote, et al. “HMDB: A Large

489

Video Database for Human Motion Recognition.” *ICCV*,

490

2011.

491

[7] E. Perez, F. Strub, H. de Vries, et al. “FiLM: Visual Reason-

492

ing with a General Conditioning Layer.” *AAAI*, 2018.

493

[8] R. Zhang, P. Isola, A. A. Efros, et al. “The Unreasonable Ef-

494

fectiveness of Deep Features as a Perceptual Metric.” *CVPR*,

495

2018.

496

[9] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All

497

You Need.” *NeurIPS*, 2017.

498

[10] P. Isola, J.-Y. Zhu, T. Zhou, et al. “Image-to-Image Transla-

499

tion with Conditional Adversarial Networks.” *CVPR*, 2017.

500

[11] O. Ronneberger, P. Fischer, T. Brox. “U-Net: Convolutional

501

Networks for Biomedical Image Segmentation.” *MICCAI*,

502

2015.

503

[12] J. Oh, X. Guo, H. Lee, et al. “Action-Conditional Video Pre-

504

diction using Deep Networks in Atari Games.” *NIPS*, 2015.

505

[13] N. Srivastava, E. Mansimov, R. Salakhutdinov. “Unsuper-

506

vised Learning of Video Representations using LSTMs.”

507

ICML, 2015.

508 [14] C. Vondrick, H. Pirsiavash, A. Torralba. “Generating Videos
509 with Scene Dynamics.” *NIPS*, 2016.
510 [15] U. Singer, A. Polyak, T. Hayes, et al. “Make-A-
511 Video: Text-to-Video Generation without Text-Video Data.”
512 *arXiv:2209.14792*, 2022.
513 [16] J. Ho, W. Chan, C. Saharia, et al. “Imagen Video:
514 High Definition Video Generation with Diffusion Models.”
515 *arXiv:2210.02303*, 2022.
516 [17] M. Monfort, A. Andonian, B. Zhou, et al. “Moments in
517 Time Dataset: One Million Videos for Event Understand-
518 ing.” *TPAMI*, 2019.
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534